```
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('universal_tagset')

sentence = 'I am a Data Science student studying at Surabaya State University'

tokens = nltk.word_tokenize(sentence)

tagged_sent = nltk.pos_tag(tokens, tagset='universal')

print(tagged_sent)
```

```
[('I', 'PRON'), ('am', 'VERB'), ('a', 'DET'), ('Data', 'NOUN'), ('Science', 'NOUN'), ('student', 'NOUN'), ('studying', 'VERB'),
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!
```

TUGAS WEEK 5

FEATURE ENGINEERING

```
import warnings
warnings.filterwarnings(action='ignore')
```

1. ONE HOT ENCODING

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

documents = [
    "karhutla di Indonesia",
    "terjadi karhulta di wilayah Sumatra dan Kalimantan",
    "Kalimantan dan Sumatra"
]

#memisahkan setiap kata dalam dokumen
words = [doc.split() for doc in documents]

#mengambil semua kata unik
all_words = sorted(set(word for doc in words for word in doc))

#mengubah kata menjadi format kolom
data = []
for doc in words:
    row = {word: 0 for word in all_words}
    for word in doc:
        row[word] = 1
    data.append(row)

df = pd.DataFrame(data)
df
```

	Indonesia	Kalimantan	Sumatra	dan	di	karhulta	karhutla	terjadi	wilayah
0	1	0	0	0	1	0	1	0	0
1	0	1	1	1	1	1	0	1	1
2	0	1	1	1	0	0	0	0	0

Langkah berikutnya: Buat kode dengan df | New interactive sheet

2. Bag of Words

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
```

```
documents = ["karhutla di Indonesia", "terjadi karhulta di wilayah Sumatra dan Kalimantan", "Kalimantan dan Sumatra"]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)
df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
df
```

	dan	di	indonesia	kalimantan	karhulta	karhutla	sumatra	terjadi	wilayah
0	0	1	1	0	0	1	0	0	0
1	1	1	0	1	1	0	1	1	1
2	1	0	0	1	0	0	1	0	0

Langkah berikutnya: Buat kode dengan df | New interactive sheet

3. TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

documents = ["karhutla di Indonesia", "terjadi karhulta di wilayah Sumatra dan Kalimantan", "Kalimantan dan Sumatra"]

# Mengubah teks menjadi matriks TF-IDF
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
df
```

	dan	di	indonesia	kalimantan	karhulta	karhutla	sumatra	terjadi	wilayah
0	0.000000	0.473630	0.622766	0.000000	0.000000	0.622766	0.000000	0.000000	0.000000
1	0.329928	0.329928	0.000000	0.329928	0.433816	0.000000	0.329928	0.433816	0.433816
2	0.577350	0.000000	0.000000	0.577350	0.000000	0.000000	0.577350	0.000000	0.000000

Langkah berikutnya: Buat kode dengan df | New interactive sheet

```
import pandas as pd

import requests
from bs4 import BeautifulSoup

page = requests.get('https://github.com/')
bs = BeautifulSoup(page.text, 'html.parser')

bs

text = bs.get_text(separator=' ', strip=True)

print(text[:5000])

df = text.split()
df = pd.DataFrame(df, columns=['prepro'])
df = df.dropna().reset_index().drop(columns=['index'])
df_pre = df['prepro']
df_pre.info()
```

```
GitHub · Change is constant. GitHub keeps you ahead. · GitHub · GitHub Skip to content Navigation Menu Sign in Platform AI CODE
<class 'pandas.core.series.Series'>
RangeIndex: 1134 entries, 0 to 1133
Series name: prepro
Non-Null Count  Dtype
--------------  -----
1134 non-null   object
dtypes: object(1)
memory usage: 9.0+ KB
```

```
# Mengubah teks menjadi matriks TF-IDF
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df_pre)
df_FE = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
df_FE
```

	12	2025	2026	24	25	3m	50	70	about	accelerate	...	with	work	workflow	workflows	world	write	you	young	yo
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
1129	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
1130	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
1131	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
1132	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
1133	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0

1134 rows × 491 columns

Tokenize Sentence

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
True
```

```
from nltk import word_tokenize
from tqdm import tqdm

sentences = [word_tokenize(tweet.lower()) for tweet in tqdm(df_pre)]
sentences
```

```
['is'],
['where'],
['you'],
['belong', '.'],
['join'],
['the'],
['world', '’', 's'],
['most'],
['widely'],
['adopted'],
['developer'],
['platform'],
['to'],
['build'],
['the'],
['technologies'],
['that'],
['shape'],
['what', '’', 's'],
['next', '.'],
...]
```

Bulid Word2Vec Model

```
!pip install gensim
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.13/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.13/dist-packages (from gensim) (2.1.3)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.13/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.13/dist-packages (from gensim) (8.0.2)
Requirement already satisfied: wrapt in /usr/local/lib/python3.13/dist-packages (from smart_open>=1.8.1->gensim) (2.5.0)
```

```
import gensim.models
# 1. Train Model
model = gensim.models.Word2Vec(sentences, min_count=1, vector_size=100, window=5, sg=0)
# 2. Save Model
model.save('word2vec_github.model')
# 3. Load Model
model = gensim.models.Word2Vec.load('word2vec_github.model')
# 4. Model Information
w2v = model.wv
w2v.index_to_key[:50]
```

```
['github',
 '.',
 'and',
 ',',
 'to',
 'your',
 'code',
 'copilot',
 'the',
 'with',
 'explore',
 'security',
 'by',
 'on',
 'for',
 'from',
 'you',
 'ai',
 'in',
 'a',
 'all',
 'customer',
 'view',
 'of',
 'support',
 'features',
 'platform',
 '’',
 'source',
 'developer',
 'that',
 'read',
 'development',
 'software',
 'industry',
 'work',
 'tools',
 'sign',
```

```
 'is',
 's',
 'about',
 'time',
 'tasks',
 'projects',
 'story',
 "'",
 'mercedes-benz',
 'together',
 'window',
 'or']
```

```
w2v.vectors.shape, w2v.vector_size # size of vector
```

```
((503, 100), 100)
```

```
model.wv.similar_by_key('github')
```

```
[('summarizes', 0.3595498502254486),
 ('then', 0.2896758019924164),
 ('workflow', 0.27538713812828064),
 ('inclusion', 0.27314773201942444),
 ('build', 0.22953389585018158),
 ('american', 0.2203543335199356),
 ('you', 0.21881335973739624),
 ('with', 0.21593932807445526),
 ('better', 0.20667937397956848),
 ('depend', 0.20454561710357666)]
```

```
model.wv.similar_by_key('code')
```

```
[('sign', 0.3488037586212158),
 ('maintainer', 0.30512794852256775),
 ('source', 0.303999662399292),
 ('accelerate', 0.2878633737564087),
 ('questions', 0.28764626383781433),
 ('cookies', 0.27864331007003784),
 ('enables', 0.256573885679245),
 ('roadmap', 0.2497168630361557),
 ('action', 0.24911832809448242),
 ('together', 0.24812301993370056)]
```

```
# 5. FE result
import numpy as np
def get_document_vector(tokens, model):
    valid_vectors = [model.wv[word] for word in tokens if word in model.wv]
    # if not valid_vectors:
    #   return np.zeros(model.vector_size)
    return np.mean(valid_vectors, axis=0)
X_features = np.array([get_document_vector(doc, model) for doc in sentences])
pd.DataFrame(X_features)
```

	0	1	2	3	4	5	6	7	8	9	...	90	
0	-0.000535	0.000234	0.005109	0.009006	-0.009307	-0.007122	0.006457	0.008973	-0.005023	-0.003760	...	0.001643	0.0001
1	0.002493	0.005999	-0.009672	-0.001414	0.005116	0.007139	0.006850	0.002580	0.006851	-0.008885	...	0.006252	-0.0031
2	-0.007615	0.000087	0.009810	0.006057	-0.005500	0.008177	-0.006448	0.009979	0.004450	0.008713	...	-0.005089	0.0037
3	-0.007776	-0.006758	-0.003154	0.006615	-0.000834	0.008884	-0.002292	-0.005304	0.003915	0.002216	...	-0.004079	-0.0082
4	-0.003636	0.005165	0.004347	0.003155	0.006087	-0.004295	0.003778	-0.001227	0.003154	-0.000161	...	0.002363	-0.0041
...	...	...	...	...	...	...	...	...	...	...	...	...	
1129	0.000194	0.002145	0.001066	0.007587	-0.004422	0.000325	0.007007	0.002252	0.004243	0.004303	...	-0.000006	0.0024
1130	-0.006442	0.000732	-0.007994	0.007562	-0.009656	-0.008284	-0.005803	-0.001763	0.000788	-0.001346	...	0.007319	0.0018
1131	0.001135	0.006175	-0.006593	0.006534	0.009794	0.003673	0.003432	0.004197	-0.001946	-0.008096	...	0.008732	0.0090
1132	0.007754	0.009374	0.006834	0.009411	-0.006880	0.008926	-0.009087	0.003586	0.005525	-0.006712	...	-0.000551	0.0022
1133	-0.007569	0.006324	0.001805	-0.001260	0.007777	-0.004582	-0.000118	0.002712	0.000004	-0.005452	...	0.005172	-0.0002

1134 rows × 100 columns

Visualize

```
!pip install umap-learn
```

```
Requirement already satisfied: umap-learn in /usr/local/lib/python3.13/dist-packages (0.5.12)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.13/dist-packages (from umap-learn) (2.1.3)
Requirement already satisfied: scipy>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from umap-learn) (1.16.3)
Requirement already satisfied: scikit-learn>=1.6 in /usr/local/lib/python3.13/dist-packages (from umap-learn) (1.6.1)
Requirement already satisfied: numba>=0.51.2 in /usr/local/lib/python3.13/dist-packages (from umap-learn) (0.61.2)
Requirement already satisfied: pynndescent>=0.5 in /usr/local/lib/python3.13/dist-packages (from umap-learn) (0.6.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-packages (from umap-learn) (4.67.3)
Requirement already satisfied: llvmlite<0.45,>=0.44.0dev0 in /usr/local/lib/python3.13/dist-packages (from numba>=0.51.2->umap-l
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.13/dist-packages (from pynndescent>=0.5->umap-learn) (1.6.
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=1.6->umap-lea
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib>=0.11->pynndescent>=0.5-
```

```
from umap import UMAP
```

```
import numpy as np
import plotly.express as px
```

```
X = UMAP().fit_transform(w2v.vectors)

df2 = pd.DataFrame(X, columns=['umap1', 'umap2'])
df2['word'] = w2v.index_to_key

fig = px.scatter(df2, x='umap1', y='umap2', text='word')
fig.update_traces(textposition='top center')
fig.update_layout(height = 800,
                  title_text = 'Word2Vec Visualization')
fig.show()
```

